



**Integrated Database and Analytics Solution for NorthStar Urban Mobility
and Logistics**

Module: Databases and Analytics

Assignment: Coursework

Student Name: Jeril Jacob

Student ID: 32146947

Submission Date: 10-05-2026

GitHub Repository Link: <https://github.com/lamJeril/northstar-databases-analytics>

Contents

1. Introduction.....	3
2. Problem Identification.....	4
3. Data Overview	5
4. Python Data Processing and Analysis	8
5. SQL within R Analysis	18
6. R Analytics and Visualisation.....	23
7. MongoDB Atlas Design and Implementation.....	29
7.1 CRUD Operations and Aggregation Queries	34
8. Query Optimisation.....	39
9. Discussion and Recommendations.....	42
10. Conclusion	45
11. References	46
12. GitHub Repository Link	47

1. Introduction

NorthStar Urban Mobility and Logistics is a sizable regional company that operates in a number of significant UK cities. Public shuttle services, last-mile deliveries, warehouse dispatch, electric car charging centers, route-planning tools, and mobile customer and driver platforms are all managed by the organization. The case study demonstrates that despite the organization's claims to be an integrated urban mobility provider, its internal systems are still disjointed and inconsistent.

The main purpose of this project is to investigate NorthStar's operational and data management problems using database and analytics techniques. The company is dealing with an increase in customer complaints, missed delivery windows, service concerns, vehicle downtime, problems with route planning, and growing operating expenses. Because crucial data is dispersed over several systems, individual reporting cannot adequately comprehend these issues.

Python data processing, SQL within R, R analytics and visualization, MongoDB Atlas NoSQL database design, CRUD operations, aggregating queries, indexing, and query optimization are all used in this research.

The objectives are to analyze hub and driver performance, find significant operational patterns, link customer complaints with delivery results, and provide a better data architecture for decision-making.

In order to complete the case study, students must combine SQL with R, R analytics, Python data processing, and MongoDB Atlas NoSQL database design into a cohesive analytical solution.

2. Problem Identification

The NorthStar case study highlights a number of interrelated organizational issues. First, the organization's data systems are dispersed. While newer services produce flexible data like app interactions, route changes, driver event streams, GPS logs, complaint histories, and platform exception records, older structured systems record orders, journeys, schedules, staffing, charging activity, maintenance transactions, warehouse dispatch events, and customer payments.

Second, NorthStar's operational performance is deteriorating. There have been more complaints, missed delivery windows, more frequent delays, and an increase in vehicle downtime. However, because related data is kept in several systems, managers are unable to pinpoint the underlying causes.

Third, the company does not have a comprehensive understanding of hub performance, route exceptions, driver behavior, vehicle condition, customer unhappiness, and service activity. Certain customer transactions might be noted as exception-handled in one system but look completed in another. Management is unable to comprehend the true reasons behind subpar service quality as a result of this uncertainty. Fourth, not all data types are appropriate for the existing relational structure. While complicated records like complaints, event histories, route exceptions, and mobile app interactions are better suited to a document-based NoSQL design, structured data like orders, deliveries, hubs, drivers, and vehicles may be analyzed using SQL.

Therefore, the main analytical challenge is not only to explain what transpired but also to pinpoint the source of inefficiencies and how data might be linked to facilitate better decision-making.

3. Data Overview

Orders, deliveries, complaints, customers, drivers, vehicles, hubs, incidents, and app events were the primary analytical datasets. A supporting reference file was the data dictionary.

File	Type	Purpose in the analysis
orders.csv	Main dataset	Service demand, customer orders, service type, pickup zone, drop-off zone, promised service window
deliveries.csv	Main dataset	Delivery status, dispatch time, completion time, drivers, vehicles, hubs, route overrides
complaints.csv	Main dataset	Complaint type, severity, customer dissatisfaction, complaint links to orders
customers.csv	Main dataset	Customer profile, customer segment, home zone, signup information
drivers.csv	Main dataset	Driver rating, training score, base zone, driver performance analysis
vehicles.csv	Main dataset	Vehicle type, battery health, mileage, vehicle condition analysis
hubs.csv	Main dataset	Hub name, hub zone, city/location-based performance analysis
incidents.csv	Main dataset	Operational disruptions, incident patterns, delivery-related problems
app_events.csv	Main dataset / NoSQL-focused	Mobile platform events, customer/driver interactions, event-based MongoDB modelling

data_dictionary.csv	Supporting dataset	Explains dataset fields, column meanings, and supports understanding of the data structure
README.txt	Supporting text file	Provides general dataset notes; not used directly in analysis

This data structure supports the assignment requirement to analyse both structured operational records and flexible semi-structured records. Structured tables are useful for SQL and R analysis, while app events, complaints, and operational histories support the need for NoSQL document modelling.

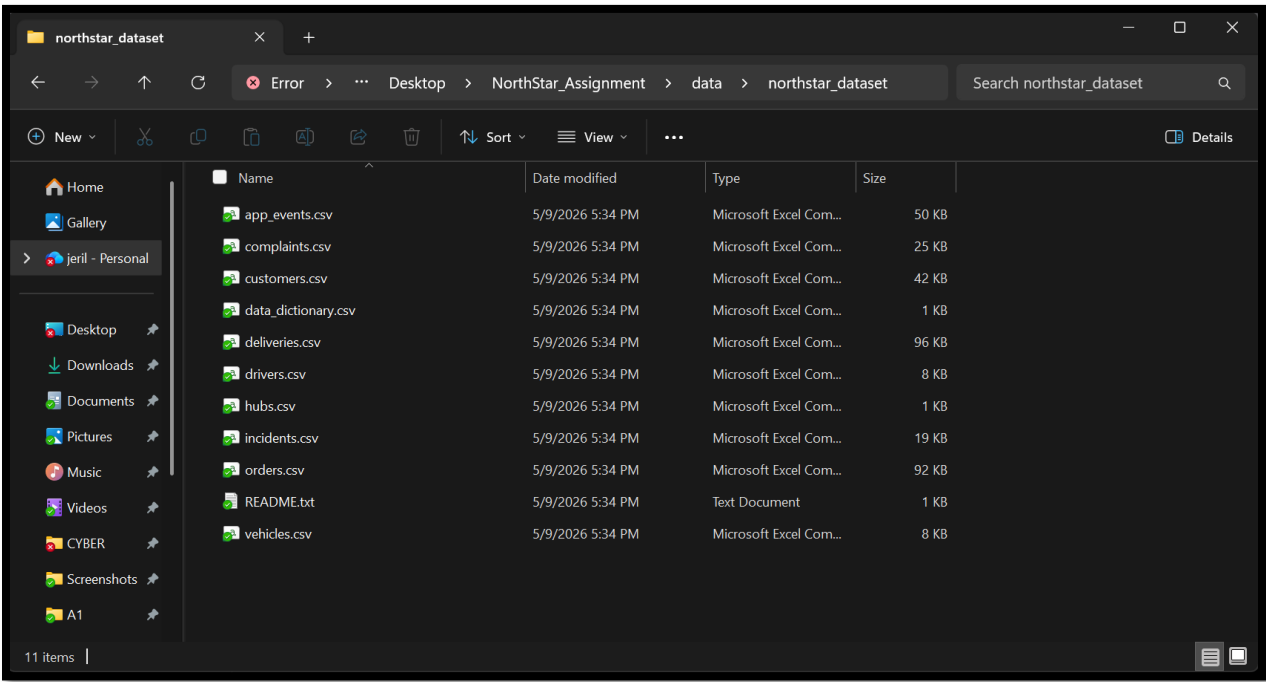


Figure 1: Dataset folder containing NorthStar files

Figure 1 shows the NorthStar dataset folder containing the CSV files used for analysis, along with the README supporting file. This confirms the source files prepared for the project workflow.

```

datasets = {
    "hubs": hubs,
    "customers": customers,
    "drivers": drivers,
    "vehicles": vehicles,
    "orders": orders,
    "deliveries": deliveries,
    "incidents": incidents,
    "complaints": complaints,
    "app_events": app_events
}

for name, df in datasets.items():
    print(f"\n{name.upper()}")
    print("Rows:", df.shape[0])
    print("Columns:", df.shape[1])
    print("Column names:", list(df.columns))

HUBS
Rows: 8
Columns: 5
Column names: ['hub_id', 'hub_name', 'zone', 'hub_type', 'capacity_score']

CUSTOMERS
Rows: 650
Columns: 9
Column names: ['customer_id', 'age', 'home_zone', 'customer_type', 'signup_date', 'loyalty_score']

DRIVERS
Rows: 170
Columns: 8
Column names: ['driver_id', 'base_zone', 'employment_type', 'years_experience', 'training_score']

```

Figure 2: Dataset summary output from Python

Figure 2 shows the dataset summary output generated in Python. This includes the number of rows, columns, and field names in each dataset, which helped identify how the files could be used for analysis and MongoDB modelling.

4. Python Data Processing and Analysis

The NorthStar datasets were cleaned, transformed, and analyzed using Python. The Python stage loaded the CSV files, checked the quality of the data, prepared analytical fields, combined related datasets, and made preliminary charts using Pandas, NumPy, and Matplotlib.

Loading all CSV files into Pandas dataframes was the initial step. Next, duplicate records and missing values were examined in the datasets. This was crucial because faulty analysis could result from missing or duplicate order, delivery, complaint, or incident records. Where necessary, location-related fields were standardized, and text fields were tidied up by eliminating extraneous spaces.

To compute the delivery period, the date and time fields were transformed to datetime format. Dispatch time and delivery completion time were compared to produce a new field called `delivery_duration_hours`. This made it possible for the research to compare delivery performance across hubs and delivery statuses and gauge how long deliveries took.

Orders, deliveries, complaints, and hubs were then combined to generate a combined analysis dataset. Given that fragmented data is NorthStar's primary problem, this was a crucial step. Customer orders, delivery results, complaint behavior, and hub performance might all be connected in a single analytical view by merging these datasets.

Delivery status, hub failure rate, average delivery time, types of complaints, manual route overrides, driver performance, and vehicle condition were the main topics of the Python study. These topics were chosen because they have a direct bearing on the case study concerns of driver performance, vehicle downtime, customer discontent, unsuccessful deliveries, route planning problems, and service delays.


```

hubs = pd.read_csv(data_path + "/hubs.csv")
customers = pd.read_csv(data_path + "/customers.csv")
drivers = pd.read_csv(data_path + "/drivers.csv")
vehicles = pd.read_csv(data_path + "/vehicles.csv")
orders = pd.read_csv(data_path + "/orders.csv")
deliveries = pd.read_csv(data_path + "/deliveries.csv")
incidents = pd.read_csv(data_path + "/incidents.csv")
complaints = pd.read_csv(data_path + "/complaints.csv")
app_events = pd.read_csv(data_path + "/app_events.csv")
data_dictionary = pd.read_csv(data_path + "/data_dictionary.csv")

print("All files loaded successfully")

... All files loaded successfully

```

Figure 3: Python libraries and CSV loading

Figure 3 shows the Python code used to load the NorthStar CSV files into Pandas dataframes. This was the starting point for the Python data processing stage and allowed the datasets to be prepared for cleaning, merging, and analysis.

```

# Check missing values in each dataset

for name, df in datasets.items():
    print(f"\n{name.upper()}")
    print(df.isnull().sum())

... dropoff_zone           0
    priority_level        0
    order_value           0
    booking_channel       25
    special_handling_flag  0
    dtype: int64

DELIVERIES
    delivery_id           0
    order_id              0
    driver_id             0
    vehicle_id            0
    hub_id                0
    dispatch_time         0
    delivery_completed_at  19
    delivery_status       0
    route_distance_km     0
    manual_route_override_count  0
    proof_of_completion_missing  0
    customer_rating_post_delivery  14
    fuel_or_charge_cost    0
    dtype: int64

```

```
INCIDENTS
... incident_id      0
   delivery_id      0
   incident_type     0
   reported_at       0
   severity          0
   resolution_status  0
   resolved_hours    17
   dtype: int64

COMPLAINTS
   complaint_id      0
   customer_id       0
   order_id          0
   complaint_type     0
   channel           0
   severity           0
   created_at         0
   status            0
   resolution_days    0
   compensation_amount 16
   dtype: int64

APP_EVENTS
   event_id          0
   customer_id       0
   order_id          144
   event_timestamp    0
   event_type         0
   session_id        0
   device_type        0
   zone_context       0
   api_latency_ms     0
   success_flag       0
   dtype: int64
```

Figure 4: Missing values check

Figure 4 shows the missing value check carried out across the datasets. This step was important because missing customer, order, delivery, complaint, or incident values

could affect the reliability of later analysis.

```
# Check duplicate records in each dataset

for name, df in datasets.items():
    duplicate_count = df.duplicated().sum()
    print(f"{name}: {duplicate_count} duplicate rows")

hubs: 0 duplicate rows
customers: 0 duplicate rows
drivers: 0 duplicate rows
vehicles: 0 duplicate rows
orders: 0 duplicate rows
deliveries: 0 duplicate rows
incidents: 0 duplicate rows
complaints: 0 duplicate rows
app_events: 0 duplicate rows
```

Figure 5: Duplicate records check

Figure 5 shows the duplicate record check. This was used to ensure that orders, deliveries, complaints, and incidents were not counted more than once during the analysis.

```
# Calculate delivery duration in hours

deliveries["delivery_duration_hours"] = (
    deliveries["delivery_completed_at"] - deliveries["dispatch_time"]
).dt.total_seconds() / 3600

deliveries[["delivery_id", "order_id", "delivery_status", "delivery_duration_hours"]].head()
```

	delivery_id	order_id	delivery_status	delivery_duration_hours
0	DL00001	O00938	Failed	22.149973
1	DL00002	O00004	OnTime	-1.100000
2	DL00003	O00639	OnTime	1.108991
3	DL00004	O00313	Delayed	23.985584
4	DL00005	O00844	OnTime	4.042814

Figure 6: Delivery duration calculation

Figure 6 shows the creation of the `delivery_duration_hours` field. This field was calculated by comparing dispatch time with delivery completion time, allowing delivery performance to be measured more accurately.

```

orders_deliveries = pd.merge(
    orders,
    deliveries,
    on="order_id",
    how="left"
)

full_analysis = pd.merge(
    orders_deliveries,
    complaints,
    on=["order_id", "customer_id"],
    how="left",
    suffixes=("", "_complaint")
)

full_analysis = pd.merge(
    full_analysis,
    hubs,
    on="hub_id",
    how="left"
)

print(full_analysis.shape)
full_analysis.head()

```

```

... (1285, 36)
order_id customer_id service_type order_created_at promised_window_hours pickup_zone dropoff_zone priority_level order_value booking_channel ... channel severity created_at status
0 O00001 C0292 Passenger 2024-08-20 14:43:00 6 Airport South Medium 126.65 App ... NaN NaN NaT NaN
1 O00002 C0459 Passenger 2024-05-14 22:16:00 24 North Airport Low 109.30 App ... NaN NaN NaT NaN
2 O00003 C0161 Passenger 2025-09-02 14:37:00 4 West Airport High 33.50 Phone ... Phone Medium 2025-09-07 14:37:00 Open
3 O00004 C0520 Parcel 2025-01-11 17:15:00 2 Riverside North Medium 10.04 App ... NaN NaN NaT NaN
4 O00005 C0558 Retail 2025-02-17 19:32:00 12 Riverside South Low 125.58 Phone ... Phone Medium 2025-02-25 19:32:00 Resolved
5 rows x 36 columns

```

resolution_days	compensation_amount	hub_name	zone	hub_type	capacity_score
NaN	NaN	North Exchange	North	Dispatch	82.0
NaN	NaN	NaN	NaN	NaN	NaN
2.0	8.66	South Link	South	Dispatch	78.0
NaN	NaN	South Link	South	Dispatch	78.0
8.0	54.41	East Dock	East	Warehouse	74.0

Figure 7: Merged analysis dataset

Figure 7 shows the merged analysis dataset created by combining related operational records. This supports the case study requirement to connect fragmented data sources and analyse service performance, complaints, and operational outcomes together.

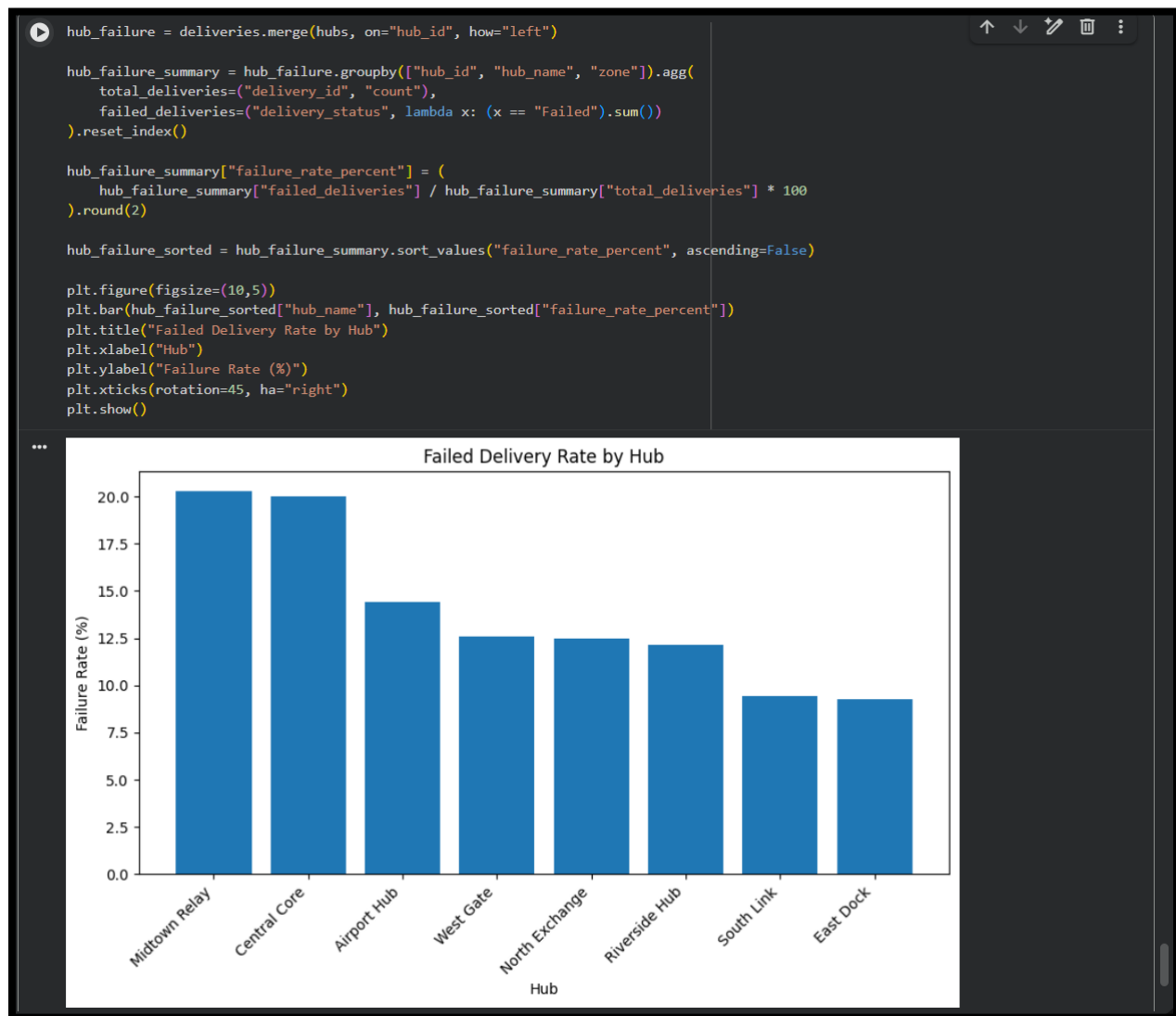


Figure 8: Failed delivery rate by hub chart

Figure 8 shows the failed delivery rate by hub. This helps identify whether service failure is concentrated in specific locations. Hubs with higher failure rates may require further investigation into route planning, staffing, vehicle availability, dispatch delays, or local operational disruption.

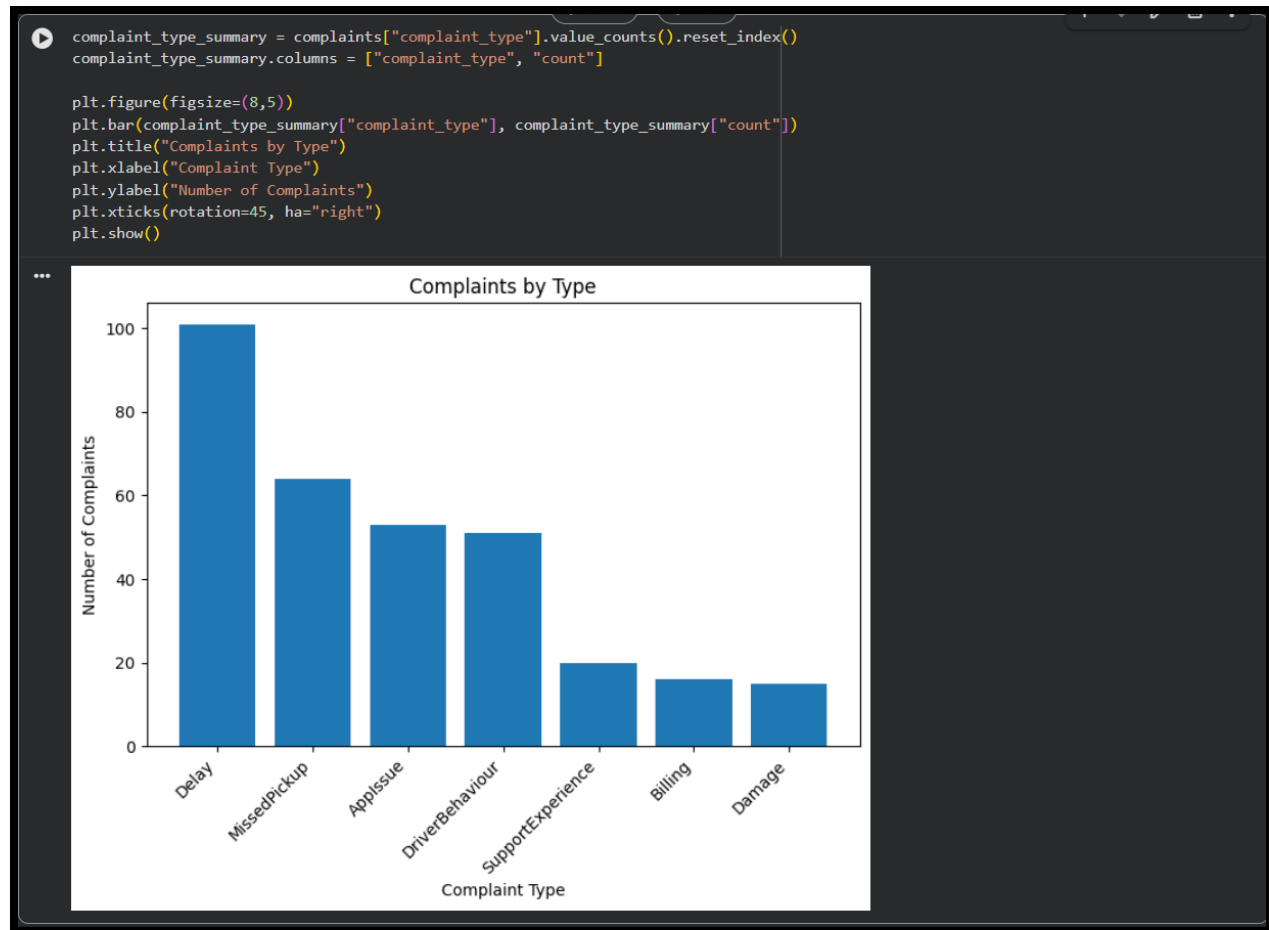


Figure 9: Complaints by type chart

Figure 9 shows the distribution of complaint types. This helps identify the main causes of customer dissatisfaction and supports NorthStar's need to connect complaints with operational performance.

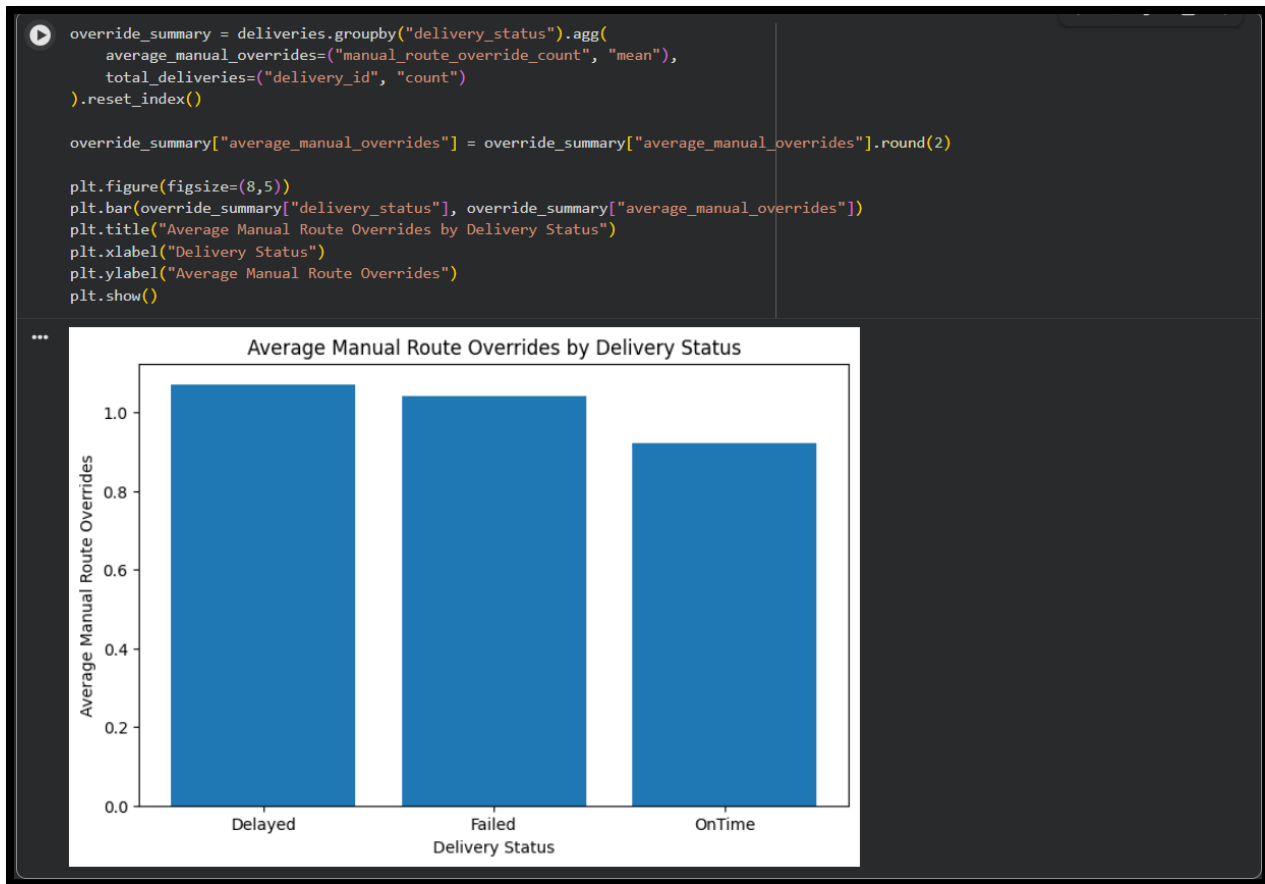


Figure 10: Manual route overrides by delivery status chart

Figure 10 compares manual route overrides across delivery statuses. Higher override levels for failed or delayed deliveries may indicate route planning issues, disruption, or driver-level changes during service execution.



Figure 11: Driver training score vs failure rate chart

Figure 11 compares driver training score with failure rate. This helps explore whether driver capability or training may contribute to poor service outcomes.

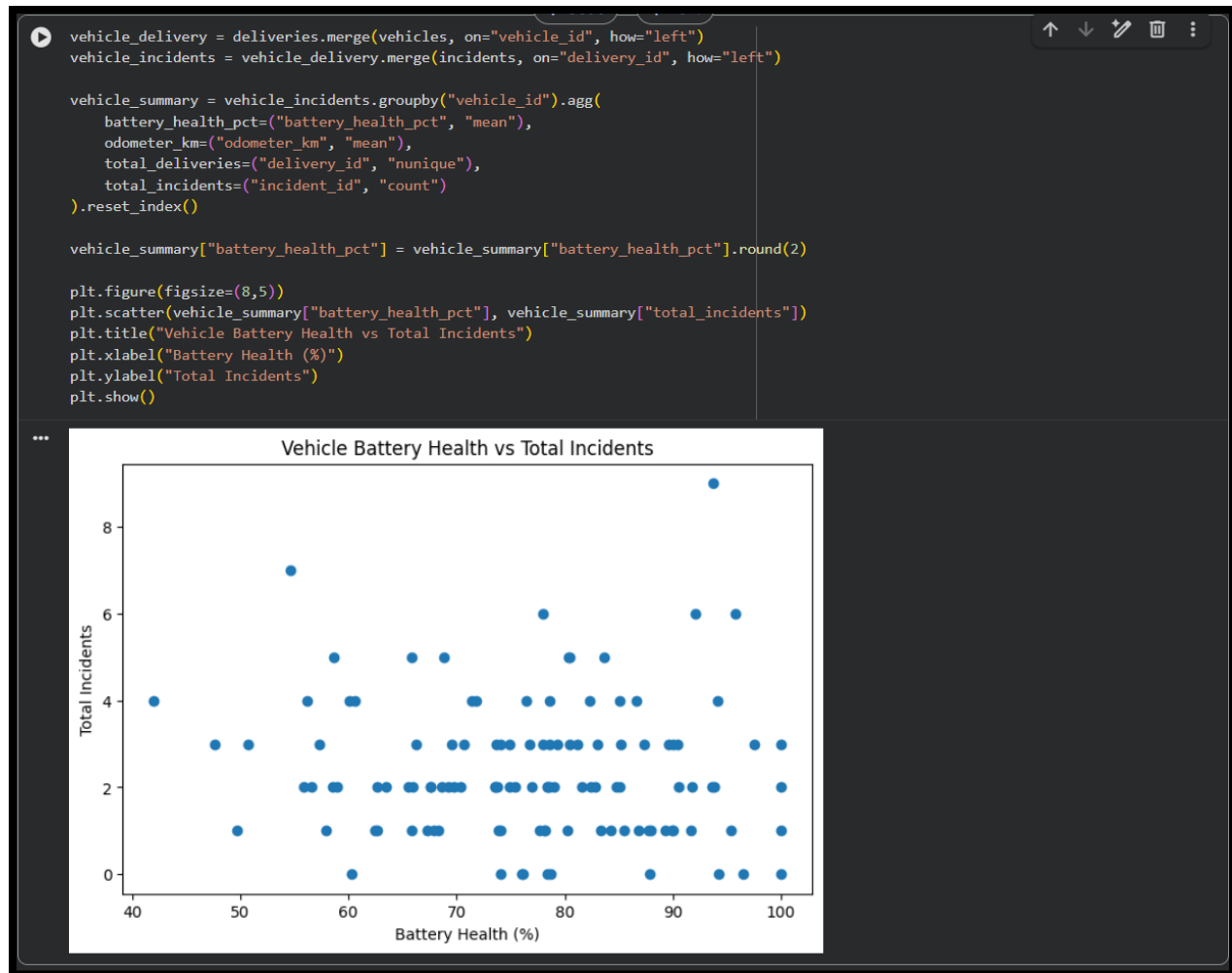


Figure 12: Vehicle battery health vs incidents chart

Figure 12 compares vehicle battery health with incident frequency. This supports the investigation of whether vehicle condition contributes to operational disruption and downtime.

5. SQL within R Analysis

The DBI and RSQLite packages were used to apply SQL within R. Structured queries within the R environment were made possible by loading the NorthStar CSV datasets into an in-memory SQLite database. This illustrated the application of basic query optimization, joins, aggregation, filtering, and SQL syntax.

Operational issues including hub performance, delivery failures, customer complaints, manual route overrides, driver performance, vehicle problems, and integrated service visibility were the main focus of the SQL study. The case study's emphasis on fragmented systems, growing complaints, missed service windows, inadequate route allocation, and increased vehicle downtime led to the selection of these areas.

To determine operational demand across several zones and hubs, the orders by hub query was utilized. Total deliveries, failed deliveries, and failure rate percentages were computed by the failed delivery query. A more equitable comparison of hubs with varying delivery volumes was made possible by the use of percentages.

The most frequent complaint categories and the service categories most frequently linked to unhappiness were found using complaint searches. Analysis of manual route overrides looked at whether deliveries that were unsuccessful or delayed were linked to more route alterations. To find out if staff capability or asset condition could be a factor in operational failures, driver and vehicle queries were employed.

Additionally, orders, deliveries, complaints, and hubs were joined to produce an integrated SQL view. Because it linked customer orders, service results, complaints, and operational location data in a single query, this directly addressed NorthStar's fragmented data issue.

```
list.files("/content")

*** 'app_events.csv' 'complaints.csv' 'customers.csv' 'data_dictionary.csv' 'deliveries.csv' 'drivers.csv' 'hubs.csv' 'incidents.csv' 'orders.csv' 'r_delivery_status_stats.csv' 'sample_data' 'sql_complaints_by_service.csv'
'sql_complaints_by_type.csv' 'sql_driver_performance.csv' 'sql_duration_by_hub.csv' 'sql_failed_by_hub.csv' 'sql_integrated_view.csv' 'sql_orders_by_hub.csv' 'sql_overrides_by_status.csv' 'sql_vehicle_incidents.csv'
'vehicles.csv'
```

Figure 13: R notebook files loaded

Figure 13 shows the NorthStar CSV files uploaded and available in the R notebook environment. This confirms that the datasets were prepared for SQL analysis within R.

```
dbListTables(con)

*** 'app_events' 'complaints' 'customers' 'deliveries' 'drivers' 'hubs' 'incidents' 'orders' 'vehicles'
```

Figure 14: SQL database tables created in R

Figure 14 shows the database tables created in the in-memory SQLite database. The CSV datasets were converted into SQL tables so that relational queries could be executed using R.

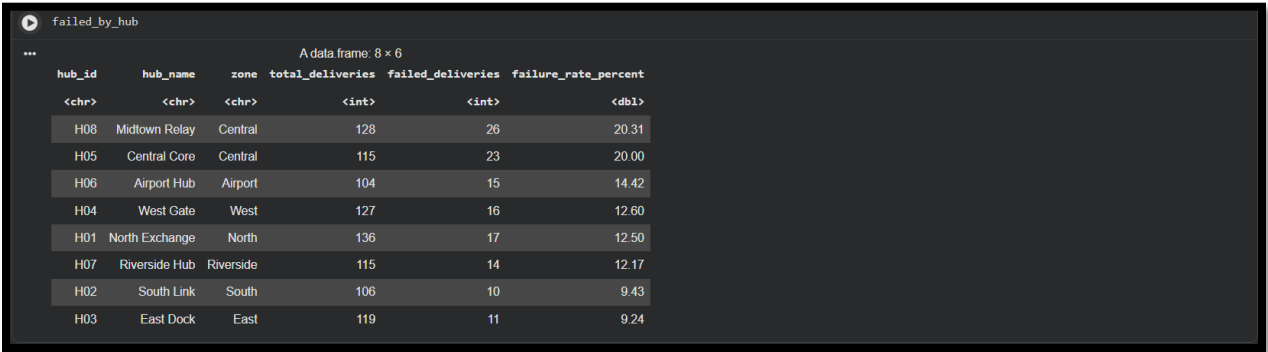
```
query_orders_by_hub <- "
SELECT
  h.hub_id,
  h.hub_name,
  h.zone,
  COUNT(o.order_id) AS total_orders
FROM orders o
LEFT JOIN hubs h
  ON LOWER(TRIM(o.pickup_zone)) = LOWER(TRIM(h.zone))
GROUP BY h.hub_id, h.hub_name, h.zone
ORDER BY total_orders DESC;
"

orders_by_hub <- dbGetQuery(con, query_orders_by_hub)
orders_by_hub

*** A data frame: 9 x 4
  hub_id hub_name zone total_orders
  <chr>   <chr>   <chr>   <int>
1 H03    East Dock East     207
2 H02    South Link South    181
3 H01    North Exchange North    174
4 H05    Central Core Central   158
5 H08    Midtown Relay Central   158
6 H04    West Gate West     155
7 H07    Riverside Hub Riverside 151
8 H06    Airport Hub Airport   144
9 NA     NA     NA      80
```

Figure 15: Orders by hub SQL output

Figure 15 shows the total number of orders by hub. This helps identify which hub zones experience the highest service demand and may therefore face greater operational pressure.

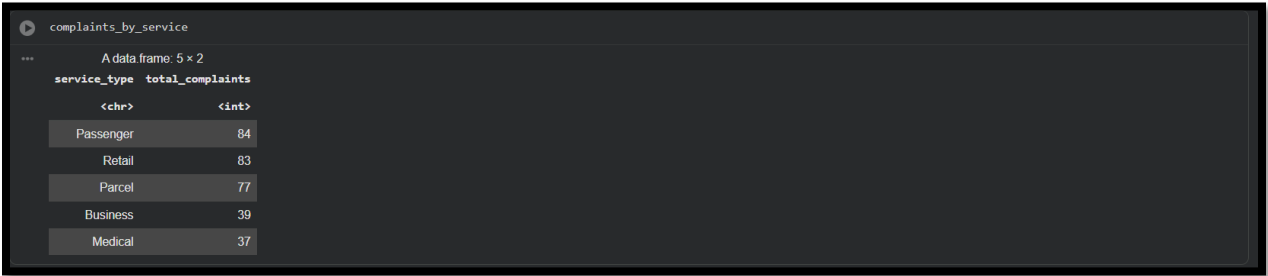


```
failed_by_hub
```

hub_id	hub_name	zone	total_deliveries	failed_deliveries	failure_rate_percent
H08	Midtown Relay	Central	128	26	20.31
H05	Central Core	Central	115	23	20.00
H06	Airport Hub	Airport	104	15	14.42
H04	West Gate	West	127	16	12.60
H01	North Exchange	North	136	17	12.50
H07	Riverside Hub	Riverside	115	14	12.17
H02	South Link	South	106	10	9.43
H03	East Dock	East	119	11	9.24

Figure 16: Failed deliveries by hub SQL output

Figure 16 shows failed deliveries by hub, including total deliveries, failed deliveries, and failure rate percentage. The failure rate percentage is useful because it allows fair comparison between hubs with different delivery volumes.



```
complaints_by_service
```

service_type	total_complaints
Passenger	84
Retail	83
Parcel	77
Business	39
Medical	37

Figure 17: Complaints by service type SQL output

Figure 17 shows complaints grouped by service type. This helps identify whether customer dissatisfaction is concentrated in passenger, parcel, retail, or other service categories

overrides_by_status		
A data frame: 3 × 3		
delivery_status	total_deliveries	average_manual_overrides
<chr>	<int>	<dbl>
Delayed	202	1.07
Failed	132	1.04
OnTime	616	0.92

Figure 18: Manual route overrides SQL output

Figure 18 compares average manual route overrides by delivery status. This is important because high manual route override levels may indicate poor route planning, real-time disruption, or driver-level route changes.

driver_performance						
A data frame: 10 × 7						
driver_id	base_zone	average_driver_rating	average_training_score	total_deliveries	failed_deliveries	failure_rate_percent
<chr>	<chr>	<dbl>	<dbl>	<int>	<int>	<dbl>
D051	WEST	3.58	75.4	2	2	100.00
D063	north	4.03	85.7	3	2	66.67
D092	East	4.24	88.2	5	3	60.00
D104	WEST	3.45	87.7	7	4	57.14
D024	RiverSide	3.35	71.4	8	4	50.00
D103	Central	4.40	72.5	4	2	50.00
D111	Airport	4.12	79.2	4	2	50.00
D132	South	4.20	77.6	4	2	50.00
D147	West	3.80	66.4	2	1	50.00
D170	West	4.48	75.2	4	2	50.00

Figure 19: Driver performance SQL output

Figure 19 shows driver performance based on delivery counts, failed deliveries, failure rate, driver rating, and training score. This helps investigate whether driver capability or training may be linked to service failure.

vehicle_incidents

A data frame: 10 x 6

vehicle_id	vehicle_type	battery_health_pct	odometer_km	total_deliveries	total_incidents
<chr>	<chr>	<dbl>	<dbl>	<int>	<int>
V047	EV	93.7	134347	17	9
V108	Diesel	54.6	141290	9	7
V030	CargoVan	78.0	134380	15	6
V046	EV	95.8	101425	10	6
V097	EV	92.1	18680	8	6
V005	CargoVan	58.6	146638	13	5
V009	CargoVan	68.8	156887	9	5
V035	CargoVan	83.6	73804	9	5
V042	EV	80.5	215870	13	5
V076	Diesel	65.8	106063	6	5

Figure 20: Vehicle incidents SQL output

Figure 20 links vehicle information with delivery and incident records. This helps examine whether vehicle condition, battery health, mileage, or vehicle type may contribute to operational incidents and downtime.

Integrated_view

A data frame: 20 x 12

order_id	customer_id	service_type	pickup_zone	dropoff_zone	delivery_status	delivery_duration_hours	manual_route_override_count	complaint_type	complaint_severity	hub_name	zone
<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<chr>	<chr>	<chr>	<chr>
O00001	C0292	Passenger	Airport	South	OnTime	2.40	2	NA	NA	North Exchange	North
O00002	C0459	Passenger	North	AIRPORT	NA	NA	NA	NA	NA	NA	NA
O00003	C0161	Passenger	West	AIRPORT	Delayed	8.86	2	ApplIssue	Medium	South Link	South
O00004	C0520	Parcel	RiverSide	North	OnTime	-1.10	1	NA	NA	South Link	South
O00005	C0558	Retail	Riverside	SOUTH	OnTime	11.70	1	MissedPickup	Medium	East Dock	East
O00006	C0437	Retail	CENTRAL	East	NA	NA	NA	NA	NA	NA	NA
O00007	C0001	Business	CENTRAL	Airport	Delayed	8.92	1	ApplIssue	High	Airport Hub	Airport
O00008	C0157	Parcel	Riverside	Riverside	OnTime	0.42	0	MissedPickup	Medium	East Dock	East
O00009	C0141	Retail	NORTH	East	OnTime	9.75	0	NA	NA	North Exchange	North
O00010	C0171	Retail	South	north	OnTime	3.00	1	NA	NA	Midtown Relay	Central
O00011	C0340	Parcel	WEST	Central	NA	NA	NA	Delay	Low	NA	NA
O00012	C0567	Business	East	Airport	OnTime	10.01	0	ApplIssue	Medium	West Gate	West
O00013	C0234	Retail	Airport	Central	OnTime	21.85	2	MissedPickup	Medium	Riverside Hub	Riverside
O00014	C0318	Passenger	CTR	CTR	OnTime	3.01	0	NA	NA	North Exchange	North
O00015	C0300	Business	SOUTH	East	OnTime	3.63	1	NA	NA	East Dock	East
O00016	C0022	Parcel	Central	CTR	OnTime	3.15	2	NA	NA	South Link	South
O00017	C0499	Parcel	AIRPORT	South	OnTime	20.73	5	NA	NA	South Link	South
O00018	C0506	Passenger	NORTH	Riverside	OnTime	3.94	0	NA	NA	Airport Hub	Airport
O00019	C0425	Passenger	North	CENTRAL	OnTime	3.31	1	NA	NA	Central Core	Central
O00020	C0249	Retail	West	AIRPORT	OnTime	-0.25	1	NA	NA	North Exchange	North

Figure 21: Integrated operational view SQL output

Figure 21 shows the integrated operational view created using SQL joins. This is one of the most important SQL outputs because it connects customer orders, service outcomes, complaints, route information, and hub-level data. This directly supports NorthStar's need to overcome fragmented reporting.

6. R Analytics and Visualisation

R was utilized to generate summary statistics and analytical visualizations from the NorthStar datasets. Finding trends across hubs, delivery results, complaints, routes, drivers, and vehicles was the goal of this phase. Charts and grouped summaries were made using the ggplot2 and dplyr tools.

Charts for total orders by hub, failed delivery rate by hub, average delivery duration by hub, complaints by type, complaints by service type, manual route overrides by delivery status, driver training score against failure rate, vehicle battery health against incidents, and delivery duration by status were all included in the R visualization stage.

The hub-level charts were useful in determining whether operational demand, delivery failure, and delivery duration were location-specific. The example study indicates that certain city zones routinely perform worse than others, which makes this crucial. The organization can find areas that need further research by visually comparing hubs.

Visualizations of complaints were useful in determining the primary reasons behind customer discontent. Because different services may have different operational issues, service-type complaint analysis also supported more focused improvement.

Visualizations of drivers and vehicles investigated potential reasons for delivery disruption and failure. Vehicle battery health was compared with total incidents, and driver training ratings were compared with failure rates. This bolstered the inquiry into whether driver capability, vehicle condition, or broader service demands could be connected to operational problems. Delivery duration by delivery status was also analyzed using a boxplot. This made it easier to see delivery completion time variance and outliers, which is helpful for spotting inconsistent service performance.



Figure 22: Total orders by hub chart

Figure 22 shows the total number of orders by hub. This helps identify which hubs experience the highest operational demand and may require greater staffing, vehicle availability, or route planning support.



Figure 23: Failed delivery rate by hub chart

Figure 23 compares hubs by failed delivery rate. This helps identify underperforming locations and supports more targeted operational improvement.



Figure 24: Average delivery duration by hub chart

Figure 24 shows average delivery duration by hub. Hubs with longer delivery durations may be affected by congestion, dispatch delays, route inefficiency, or operational pressure.



Figure 25: Complaints by type chart

Figure 25 shows complaints grouped by complaint type. This helps identify the most common causes of customer dissatisfaction and supports service quality improvement.



Figure 26: Complaints by service type chart

Figure 26 shows complaints by service type. This helps identify whether dissatisfaction is concentrated in specific services, allowing NorthStar to target improvements more effectively.



Figure 27: Manual route overrides by delivery status chart

Figure 27 compares average manual route overrides by delivery status. Higher override values for delayed or failed deliveries may indicate route planning issues, real-time disruption, or driver-level route changes.

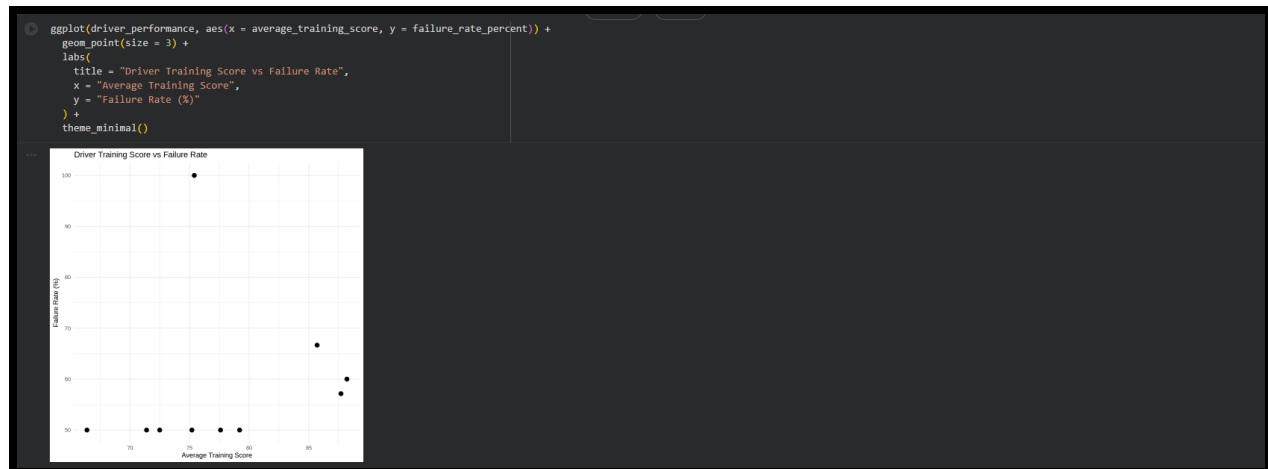


Figure 28: Driver training score vs failure rate chart

Figure 28 compares driver training score with delivery failure rate. This helps explore whether driver training and capability may influence service outcomes.

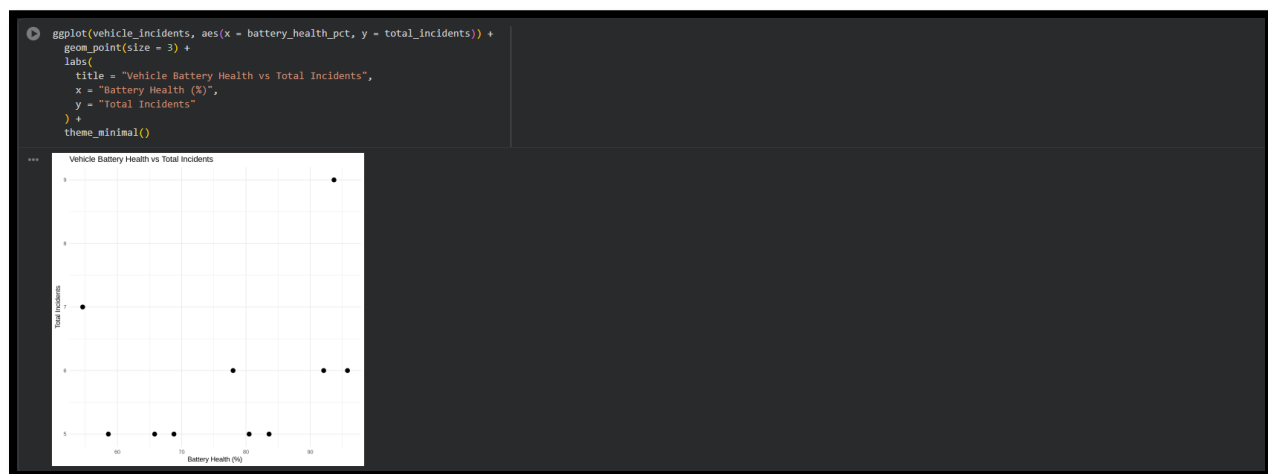


Figure 29: Vehicle battery health vs incidents chart

Figure 29 compares vehicle battery health with total incidents. This helps examine whether weaker vehicle condition may be linked with more operational disruption.

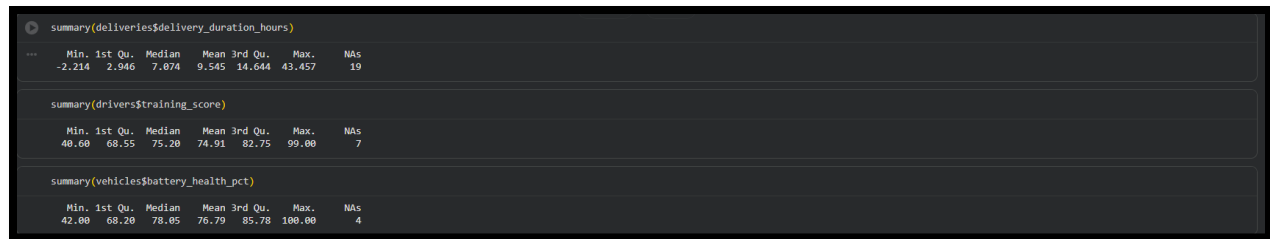


Figure 30: Delivery duration summary statistics

Figure 30 shows summary statistics for delivery duration and related performance fields. These statistics provide a basic understanding of the distribution, average values, and possible variation in the operational data.

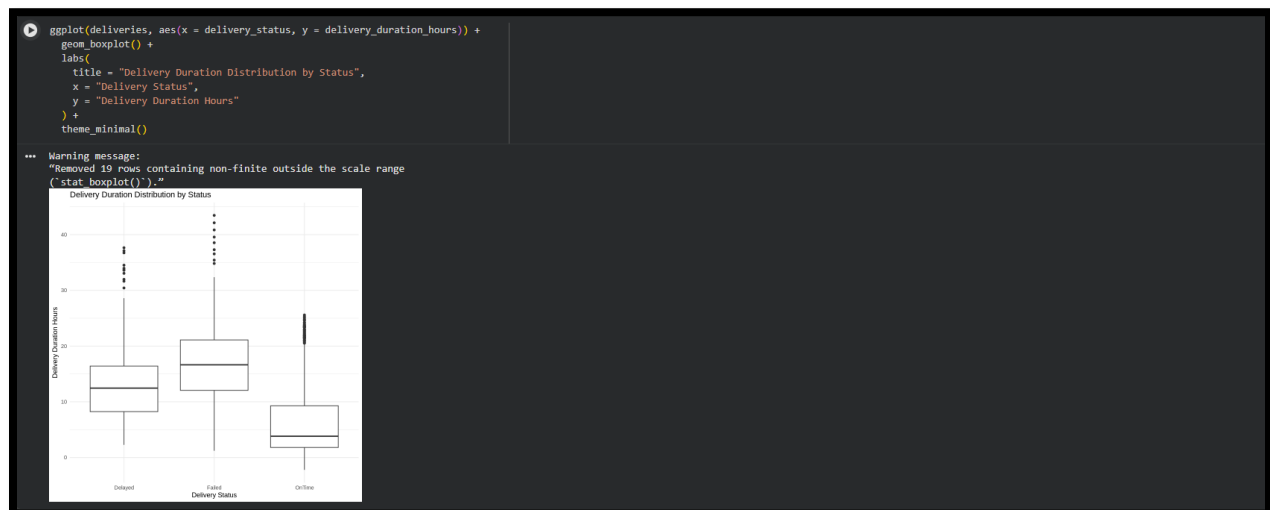


Figure 31: Delivery duration distribution by status boxplot

Figure 31 shows the distribution of delivery duration by delivery status. The boxplot helps identify variation and outliers, which may indicate inconsistent service performance or unusual operational delays.

7. MongoDB Atlas Design and Implementation

For NorthStar Urban Mobility and Logistics, a NoSQL database solution was designed and implemented using MongoDB Atlas. This portion of the project concentrated on records like customer case histories, service events, route exceptions, complaints, and app interactions that are challenging to maintain with just conventional relational tables. According to the case study, mobile app interactions, driver event streams, route modifications, customer complaints, service chat histories, GPS path logs, and platform exception records are just a few examples of the versatile and complicated data that NorthStar's more recent services provide. These records are appropriate for a document-based database design since they are nested, changeable, and quickly changing.

Northstar_db is the name of the MongoDB Atlas database.

Customer_cases, service_events, route_exceptions, and app_interactions were the four primary collections utilized. An integrated operational view of customers, deliveries, complaints, events, and exceptions was intended to be supported by these collections.

Customer-level documents were stored in the customer_cases collection. A client profile, associated orders, delivery records, complaint history, and a case summary were all included in each document. Instead of looking across many systems, this approach enables NorthStar to observe consumer activity and dissatisfaction in one location.

Delivery-level operational records were kept in the service_events collection. Delivery details, driver details, vehicle details, hub details, and associated occurrences were all recorded in each document. Because service outcomes may be connected to hub-level activities, vehicle condition, and driver performance, this approach facilitates operational analysis.

Deliveries with manual route overrides were kept in the route_exceptions collection. Because the case study raises issues about abnormally high numbers of manual route alterations, this collection is helpful. NorthStar is able to analyze route disruption and compare it with delivery outcomes by storing route exceptions individually. Mobile platform event records were kept in the app_interactions collection. The kind of adaptable event data produced by driver and customer apps is reflected in this collection. Because event records can have different structures and contain fields that change over time, MongoDB is a good fit for this kind of data.

Collection	Purpose	Main data included
customer_cases	Customer-level operational view	Customer profile, orders, deliveries, complaints, case summary
service_events	Delivery-level operational tracking	Delivery status, driver, vehicle, hub, incidents
route_exceptions	Route disruption analysis	Manual route overrides, delivery status, order and hub details
app_interactions	Platform event storage	Customer app events, driver app events, event type, timestamp

In this style, related information is frequently accessed simultaneously through embedded documents. For instance, because managers might need to examine client history as a single, comprehensive case, customer orders and complaints were integrated into customer case papers. In a similar vein, service event documents included information about the driver, vehicle, hub, and incident to provide operational analysis of every delivery. This approach facilitates quicker case-level retrieval and lessens the need for repeated joins.

```
import os

data_path = "/content"

os.listdir(data_path)

... ['config',
      'deliveries.csv',
      'orders.csv',
      'hubs.csv',
      'customers.csv',
      'app_events.csv',
      'data_dictionary.csv',
      'complaints.csv',
      'ipynb_checkpoints',
      'incidents.csv',
      'vehicles.csv',
      'drivers.csv',
      'sample_data']
```

Figure 32: CSV files uploaded into MongoDB Colab notebook

Figure 32 shows the NorthStar CSV files uploaded into the MongoDB Colab notebook. This was required so that the datasets could be loaded using Pandas and transformed into MongoDB document structures.

```
orders = pd.read_csv("/content/orders.csv")
deliveries = pd.read_csv("/content/deliveries.csv")
complaints = pd.read_csv("/content/complaints.csv")
customers = pd.read_csv("/content/customers.csv")
drivers = pd.read_csv("/content/drivers.csv")
vehicles = pd.read_csv("/content/vehicles.csv")
hubs = pd.read_csv("/content/hubs.csv")
incidents = pd.read_csv("/content/incidents.csv")
app_events = pd.read_csv("/content/app_events.csv")

print("All CSV files loaded successfully")

... All CSV files loaded successfully
```

Figure 33: CSV files loaded successfully

Figure 33 shows the CSV files being loaded successfully into the MongoDB notebook. This confirms that the data was ready for transformation into MongoDB collections.

```

customer_documents = []
for _, customer in customers.iterrows():
    customer_id = customer["customer_id"]

    customer_orders = orders[orders["customer_id"] == customer_id]
    customer_complaints = complaints[complaints["customer_id"] == customer_id]

    order_list = []

    for _, order in customer_orders.iterrows():
        order_id = order["order_id"]

        delivery_records = deliveries[deliveries["order_id"] == order_id]
        complaint_records = complaints[complaints["order_id"] == order_id]

        delivery_list = delivery_records.to_dict("records")
        complaint_list = complaint_records.to_dict("records")

        order_doc = order.to_dict()
        order_doc["deliveries"] = delivery_list
        order_doc["complaints"] = complaint_list

        order_list.append(order_doc)

    customer_doc = {
        "customer_id": customer_id,
        "customer_profile": customer.to_dict(),
        "orders": order_list,
        "complaint_history": customer_complaints.to_dict("records"),
        "case_summary": {
            "total_orders": len(customer_orders),
            "total_complaints": len(customer_complaints)
        }
    }

    customer_documents.append(customer_doc)

print("Customer case documents created:", len(customer_documents))

```

Customer case documents created: 650

Figure 34: Customer case documents created

Figure 34 shows the creation of customer case documents. These documents combine customer profiles, orders, deliveries, and complaint history into one customer-level structure.

```

if customer_documents:
    result = customer_cases.insert_many(customer_documents)
    print("Inserted customer case documents:", len(result.inserted_ids))
else:
    print("No customer documents to insert")

Inserted customer case documents: 650

```

Figure 35: Customer case documents inserted

Figure 35 shows the insertion of customer case documents into the `customer_cases` collection. This demonstrates how relational records were transformed into nested MongoDB documents.


```

service_event_documents = []

for _, delivery in deliveries.iterrows():
    driver_info = drivers[drivers["driver_id"] == delivery["driver_id"]]
    vehicle_info = vehicles[vehicles["vehicle_id"] == delivery["vehicle_id"]]
    hub_info = hubs[hubs["hub_id"] == delivery["hub_id"]]
    incident_records = incidents[incidents["delivery_id"] == delivery["delivery_id"]]

    service_doc = {
        "delivery_id": delivery["delivery_id"],
        "order_id": delivery["order_id"],
        "delivery_status": delivery["delivery_status"],
        "dispatch_time": delivery.get("dispatch_time"),
        "delivery_completed_at": delivery.get("delivery_completed_at"),
        "manual_route_override_count": delivery.get("manual_route_override_count"),
        "customer_rating_post_delivery": delivery.get("customer_rating_post_delivery"),
        "driver": driver_info.iloc[0].to_dict() if len(driver_info) > 0 else None,
        "vehicle": vehicle_info.iloc[0].to_dict() if len(vehicle_info) > 0 else None,
        "hub": hub_info.iloc[0].to_dict() if len(hub_info) > 0 else None,
        "incidents": incident_records.to_dict("records")
    }

    service_event_documents.append(service_doc)

print("Service event documents created:", len(service_event_documents))

if service_event_documents:
    result = service_events.insert_many(service_event_documents)
    print("Inserted service event documents:", len(result.inserted_ids))
else:
    print("No service event documents to insert")

Service event documents created: 950
Inserted service event documents: 950

```

Figure 36: Service event documents inserted

Figure 36 shows service event documents being created and inserted into MongoDB. These documents connect delivery outcomes with driver, vehicle, hub, and incident information.

```

route_exception_documents = []

for _, delivery in deliveries.iterrows():
    override_count = delivery.get("manual_route_override_count")

    if override_count is not None and override_count > 0:
        order_info = orders[orders["order_id"] == delivery["order_id"]]
        hub_info = hubs[hubs["hub_id"] == delivery["hub_id"]]

        route_doc = {
            "delivery_id": delivery["delivery_id"],
            "order_id": delivery["order_id"],
            "delivery_status": delivery["delivery_status"],
            "manual_route_override_count": override_count,
            "dispatch_time": delivery.get("dispatch_time"),
            "delivery_completed_at": delivery.get("delivery_completed_at"),
            "order": order_info.iloc[0].to_dict() if len(order_info) > 0 else None,
            "hub": hub_info.iloc[0].to_dict() if len(hub_info) > 0 else None
        }

        route_exception_documents.append(route_doc)

print("Route exception documents created:", len(route_exception_documents))

if route_exception_documents:
    result = route_exceptions.insert_many(route_exception_documents)
    print("Inserted route exception documents:", len(result.inserted_ids))
else:
    print("No route exception documents to insert")

Route exception documents created: 551
Inserted route exception documents: 551

```

Figure 37: Route exception documents inserted

Figure 37 shows route exception documents inserted into MongoDB. These documents focus on deliveries with manual route overrides, supporting analysis of route disruption and operational exceptions.

```

app_interaction_documents = app_events.to_dict("records")

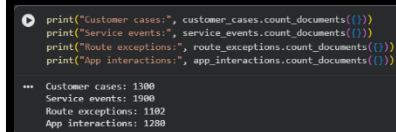
if app_interaction_documents:
    result = app_interactions.insert_many(app_interaction_documents)
    print("Inserted app interaction documents:", len(result.inserted_ids))
else:
    print("No app interaction documents to insert")

Inserted app interaction documents: 640

```

Figure 38: App interaction documents inserted

Figure 38 shows app interaction documents inserted into MongoDB. This collection supports storage of flexible platform event data generated by customer and driver applications.



```
print("Customer cases:", customer_cases.count_documents({}))
print("Service events:", service_events.count_documents({}))
print("Route exceptions:", route_exceptions.count_documents({}))
print("App interactions:", app_interactions.count_documents({}))

... Customer cases: 1380
Service events: 1500
Route exceptions: 1182
App interactions: 1280
```

Figure 39: Final MongoDB document counts

Figure 39 confirms that the MongoDB collections were populated successfully. The document counts show that customer cases, service events, route exceptions, and app interactions were inserted into MongoDB Atlas.

7.1 CRUD Operations and Aggregation Queries

PyMongo was used in Google Colab to implement CRUD operations. A demo customer case document was added to the customer_cases collection as part of a create operation. The inserted document was subsequently retrieved via a read operation using find_one(). After testing, a delete action deleted the demo record and an update operation updated the complaint status from Open to Resolved.

These actions show how operational case management can be supported by MongoDB. For instance, the customer case document can be updated to reflect the resolution of a customer complaint. This bolsters NorthStar's demand for more adaptable operational records and case-level tracking.

Additionally, aggregation queries were put into practice. One aggregation displayed the number of consumers with varying levels of complaint history by grouping customer instances according to the total number of complaints. Another aggregation computed average route overrides and customer ratings by grouping service events according to delivery status. In order to pinpoint areas with greater instances of route disruption, a third aggregation grouped route exceptions by hub.

These aggregation queries demonstrate MongoDB's ability to handle both operational storage and analytical use cases. NorthStar can benefit from this since it needs to use more adaptable data structures to analyze hub-level performance, delivery results, route modifications, and customer unhappiness.

```
new_customer_case = {
  "customer_id": "C_DEMO_001",
  "customer_profile": {
    "customer_id": "C_DEMO_001",
    "customer_segment": "Business",
    "home_zone": "Central",
    "signup_date": "2026-05-10"
  },
  "orders": [
    {
      "order_id": "O_DEMO_001",
      "service_type": "Parcel",
      "pickup_zone": "Central",
      "dropoff_zone": "Airport",
      "delivery_status": "Delayed"
    }
  ],
  "complaint_history": [
    {
      "complaint_id": "CMP_DEMO_001",
      "complaint_type": "Late Delivery",
      "severity": "High",
      "status": "Open"
    }
  ],
  "case_summary": {
    "total_orders": 1,
    "total_complaints": 1
  }
}

insert_result = customer_cases.insert_one(new_customer_case)

print("Inserted demo customer case ID:", insert_result.inserted_id)

*** Inserted demo customer case ID: 6a00529fdabf0406614dcb27
```

Figure 40: MongoDB create operation

Figure 40 shows a create operation where a demo customer case document was inserted into the customer_cases collection. This demonstrates how MongoDB can add new customer case records.

```

demo_customer = customer_cases.find_one({"customer_id": "C_DEMO_001"})

pprint.pprint(demo_customer)

'''
{'_id': ObjectId('6a00529fdabf0406614dcb27'),
 'case_summary': {'total_complaints': 1, 'total_orders': 1},
 'complaint_history': [{'complaint_id': 'CMP_DEMO_001',
                        'complaint_type': 'Late Delivery',
                        'severity': 'High',
                        'status': 'Open'}],
 'customer_id': 'C_DEMO_001',
 'customer_profile': {'customer_id': 'C_DEMO_001',
                      'customer_segment': 'Business',
                      'home_zone': 'Central',
                      'signup_date': '2026-05-10'},
 'orders': [{'delivery_status': 'Delayed',
              'dropoff_zone': 'Airport',
              'order_id': 'O_DEMO_001',
              'pickup_zone': 'Central',
              'service_type': 'Parcel'}]}
'''

```

Figure 41: MongoDB read operation

Figure 41 shows a read operation using `find_one()` to retrieve the demo customer case document. This demonstrates how MongoDB can retrieve nested customer case records from the collection.

```

update_result = customer_cases.update_one(
    {"customer_id": "C_DEMO_001"},
    {
        "$set": {
            "complaint_history.0.status": "Resolved",
            "case_summary.last_updated": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        }
    }
)

print("Matched documents:", update_result.matched_count)
print("Modified documents:", update_result.modified_count)

'''
Matched documents: 1
Modified documents: 1
'''

updated_customer = customer_cases.find_one(
    {"customer_id": "C_DEMO_001"},
    {"customer_id": 1, "complaint_history": 1, "case_summary": 1}
)

pprint.pprint(updated_customer)

{'_id': ObjectId('6a00529fdabf0406614dcb27'),
 'case_summary': {'last_updated': '2026-05-10 09:41:57',
                  'total_complaints': 1,
                  'total_orders': 1},
 'complaint_history': [{'complaint_id': 'CMP_DEMO_001',
                        'complaint_type': 'Late Delivery',
                        'severity': 'High',
                        'status': 'Resolved'}],
 'customer_id': 'C_DEMO_001'}
'''

```

Figure 42: MongoDB update operation

Figure 42 shows an update operation where the complaint status in the demo customer case was changed from Open to Resolved. This demonstrates how MongoDB can update nested case information.

```

delete_result = customer_cases.delete_one({"customer_id": "C_DEMO_001"})

print("Deleted documents:", delete_result.deleted_count)

Deleted documents: 1

```

Figure 43: MongoDB delete operation

Figure 43 shows a delete operation where the demo customer case document was removed from the collection after testing. This confirms that the full CRUD cycle was completed.

```

complaint_summary = customer_cases.aggregate([
    {
        "$group": {
            "_id": "$case_summary.total_complaints",
            "number_of_customers": {"$sum": 1}
        },
    },
    {
        "$sort": {"_id": -1}
    }
])

for item in complaint_summary:
    pprint.pprint(item)

{'_id': 4, 'number_of_customers': 2}
{'_id': 3, 'number_of_customers': 22}
{'_id': 2, 'number_of_customers': 124}
{'_id': 1, 'number_of_customers': 318}
{'_id': 0, 'number_of_customers': 834}

```

Figure 44: Complaint aggregation output

Figure 44 shows an aggregation query grouping customer cases by total complaint count. This helps identify how many customers have different levels of complaint history.

```

delivery_status_summary = service_events.aggregate([
    {
        "$group": {
            "_id": "$delivery_status",
            "total_deliveries": {"$sum": 1},
            "average_route_overrides": {"$avg": "$manual_route_override_count"},
            "average_customer_rating": {"$avg": "$customer_rating_post_delivery"}
        },
    },
    {
        "$sort": {"total_deliveries": -1}
    }
])

for item in delivery_status_summary:
    pprint.pprint(item)

{'_id': 'Online',
 'average_customer_rating': nan,
 'average_route_overrides': 0.9204545454545454,
 'total_deliveries': 1232}
{'_id': 'Delayed',
 'average_customer_rating': nan,
 'average_route_overrides': 1.0742574257425743,
 'total_deliveries': 404}
{'_id': 'Failed',
 'average_customer_rating': nan,
 'average_route_overrides': 1.0378787878787878,
 'total_deliveries': 264}

```

Figure 45: Delivery status aggregation output

Figure 45 shows an aggregation query grouping service events by delivery status. It also calculates average route overrides and customer ratings, helping connect delivery outcomes with service quality indicators.

```

route_exception_by_hub = route_exceptions.aggregate([
    {
        "$group": {
            "_id": "$hub.hub_name",
            "total_route_exceptions": {"$sum": 1},
            "average_overrides": {"$avg": "$manual_route_override_count"}
        },
        {"$sort": {"total_route_exceptions": -1}}
    ])

for item in route_exception_by_hub:
    pprint.pprint(item)

{'_id': 'Midtown Relay',
 'average_overrides': 1.6904761904761905,
 'total_route_exceptions': 168}
{'_id': 'North Exchange',
 'average_overrides': 1.6867469879518073,
 'total_route_exceptions': 166}
{'_id': 'West Gate',
 'average_overrides': 1.5633802816901408,
 'total_route_exceptions': 142}
{'_id': 'Central Core',
 'average_overrides': 1.676923076923077,
 'total_route_exceptions': 130}
{'_id': 'East Dock',
 'average_overrides': 1.6825396825396826,
 'total_route_exceptions': 126}
{'_id': 'Riverside Hub',
 'average_overrides': 1.9206349206349207,
 'total_route_exceptions': 126}
{'_id': 'South Link',
 'average_overrides': 1.564516129032258,
 'total_route_exceptions': 124}
{'_id': 'Airport Hub',
 'average_overrides': 1.5833333333333333,
 'total_route_exceptions': 120}

```

Figure 46: Route exception aggregation output

Figure 46 shows route exceptions grouped by hub. This helps identify locations where manual route overrides occur more frequently and supports investigation into route planning issues.

8. Query Optimisation

Both SQL and MongoDB query optimization were shown. Indexes on often used fields like order_id, hub_id, driver_id, and vehicle_id were established at the SQL within R stage. Indexing these fields increases query efficiency because they were frequently utilized in joins and filters.

Fields that are expected to be commonly utilized in operational searches and analytics were the basis for the creation of indexes in MongoDB. The customer_cases collection was indexed on customer_id and case_summary.total_complaints. Delivery_id, order_id, and delivery_status were used to index the service_events collection. Delivery_id and manual_route_override_count were used to index the route_exceptions collection. Customer_id and event_type were used to index the app_interactions collection.

Indexing is crucial since it makes it easier for the database to find relevant documents. In the absence of indexes, MongoDB can have to search through numerous documents for relevant records. Indexes make it easier to perform queries that search by customer ID, delivery status, route override count, or event type.

To see how SQL and MongoDB intended to carry out specific queries, explain plans were utilized. Explain output aids in demonstrating whether an effective query path is being used by the database. NorthStar needs to handle growing amounts of customer, delivery, route, and app interaction data, therefore this is crucial. Indexing and query optimization become crucial for sustaining acceptable performance as data volumes increase.

```
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_orders_order_id ON orders(order_id);")
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_deliveries_order_id ON deliveries(order_id);")
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_complaints_order_id ON complaints(order_id);")
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_deliveries_hub_id ON deliveries(hub_id);")
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_deliveries_driver_id ON deliveries(driver_id);")
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_deliveries_vehicle_id ON deliveries(vehicle_id);")

print("Indexes created successfully")

0
0
0
0
0
0
0
[1] "Indexes created successfully"
```

Figure 47: SQL indexes created

Figure 47 shows SQL indexes being created in the R SQLite database. Indexes were created on fields commonly used in joins and filters, such as order_id, hub_id, driver_id, and vehicle_id.

query_plan				
A data frame: 3 x 4				
id	parent	notused	detail	
<int>	<int>	<int>	<chr>	
7	0	61	SEARCH d USING INDEX idx_deliveries_hub_id (hub_id=?)	
12	0	61	SEARCH o USING INDEX idx_orders_order_id (order_id=?)	
18	0	61	SEARCH c USING INDEX idx_complaints_order_id (order_id=?) LEFT-JOIN	

Figure 48: SQL explain query plan

Figure 48 shows the SQL explain query plan output. This was used to inspect how the SQL engine planned to execute the query and to demonstrate that query performance was considered.

```
customer_cases.create_index("customer_id")
customer_cases.create_index("case_summary.total_complaints")

service_events.create_index("delivery_id")
service_events.create_index("order_id")
service_events.create_index("delivery_status")

route_exceptions.create_index("delivery_id")
route_exceptions.create_index("manual_route_override_count")

app_interactions.create_index("customer_id")
app_interactions.create_index("event_type")

print("Indexes created successfully")

Indexes created successfully
```

Figure 49: MongoDB indexes created

Figure 49 shows the MongoDB indexes being created using PyMongo. Indexes were created on important fields such as customer_id, delivery_status, manual_route_override_count, and event_type.


```
print("Customer cases indexes:")
pprint.pprint(list(customer_cases.list_indexes()))

print("Service events indexes:")
pprint.pprint(list(service_events.list_indexes()))

print("Route exceptions indexes:")
pprint.pprint(list(route_exceptions.list_indexes()))

print("App interactions indexes:")
pprint.pprint(list(app_interactions.list_indexes()))

--- Customer cases indexes:
[SON([('v', 2), ('key', SON([('id', 1)]))], ('name', 'id'))],
SON([('v', 2), ('key', SON([('customer_id', 1)]))], ('name', 'customer_id_1')),
SON([('v', 2), ('key', SON([('case_summary.total_complaints', 1)]))], ('name', 'case_summary.total_complaints_1'))]

Service events indexes:
[SON([('v', 2), ('key', SON([('id', 1)]))], ('name', 'id'))],
SON([('v', 2), ('key', SON([('delivery_id', 1)]))], ('name', 'delivery_id_1')),
SON([('v', 2), ('key', SON([('order_id', 1)]))], ('name', 'order_id_1')),
SON([('v', 2), ('key', SON([('delivery_status', 1)]))], ('name', 'delivery_status_1'))]

Route exceptions indexes:
[SON([('v', 2), ('key', SON([('id', 1)]))], ('name', 'id'))],
SON([('v', 2), ('key', SON([('delivery_id', 1)]))], ('name', 'delivery_id_1')),
SON([('v', 2), ('key', SON([('manual_route_override_count', 1)]))], ('name', 'manual_route_override_count_1'))]

App interactions indexes:
[SON([('v', 2), ('key', SON([('id', 1)]))], ('name', 'id'))],
SON([('v', 2), ('key', SON([('customer_id', 1)]))], ('name', 'customer_id_1')),
SON([('v', 2), ('key', SON([('event_type', 1)]))], ('name', 'event_type_1'))]
```

Figure 50: MongoDB list indexes output

Figure 50 confirms that indexes were created across all MongoDB collections. These indexes support faster retrieval for common operational queries such as searching by customer ID, delivery status, route override count, and event type.

```
explain_result = customer_cases.find(
    {"customer_id": "C001"}
).explain()

pprint.pprint(explain_result["queryPlanner"]["winningPlan"])

{'inputStage': {'direction': 'forward',
  'indexBounds': {'customer_id': {'["C001", "C001"]'}},
  'indexName': 'customer_id_1',
  'indexVersion': 2,
  'isMultiKey': False,
  'isPartial': False,
  'isSparse': False,
  'isUnique': False,
  'keyPattern': {'customer_id': 1},
  'multiKeyPaths': {'customer_id': []},
  'stage': 'EXSCAN'},
  'isCached': False,
  'stage': 'FETCH'}
```

Figure 51: MongoDB explain plan for customer query

Figure 51 shows the MongoDB explain plan for a customer_id query. This helps demonstrate how MongoDB planned to search the customer_cases collection using an indexed field.

```
explain_service_status = service_events.find(
    {"delivery_status": "Failed"}
).explain()

pprint.pprint(explain_service_status["queryPlanner"]["winningPlan"])

{'inputStage': {'direction': 'forward',
  'indexBounds': {'delivery_status': {'["Failed", "Failed"]'}},
  'indexName': 'delivery_status_1',
  'indexVersion': 2,
  'isMultiKey': False,
  'isPartial': False,
  'isSparse': False,
  'isUnique': False,
  'keyPattern': {'delivery_status': 1},
  'multiKeyPaths': {'delivery_status': []},
  'stage': 'EXSCAN'},
  'isCached': False,
  'stage': 'FETCH'}
```

Figure 52: MongoDB explain plan for delivery status query

Figure 52 shows the MongoDB explain plan for a delivery_status query. This supports the optimisation requirement because it demonstrates that query performance was reviewed rather than only inserting and retrieving data.

9. Discussion and Recommendations

The analysis demonstrates that NorthStar's operational issues are interconnected rather than isolated. It is necessary to analyze delivery failures, customer complaints, manual route overrides, driver performance, vehicle condition, and hub-level variances collectively. This bolsters the case study's claim that management finds it challenging to pinpoint the underlying reasons of fragmented data systems.

Delivery outcomes, complaints, hub performance, route overrides, driver performance, and vehicle-related interruption can all be investigated using structured operational data, as demonstrated by the Python, SQL, and R study. Python helped with dataset transformation, cleaning, and merging. For combining structured tables and building integrated operational views, R's SQL was helpful. Patterns among hubs, service kinds, complaints, route overrides, and delivery statuses were presented with the use of R visualization.

A better framework for adaptable and nested operational records was offered by MongoDB Atlas. Orders, delivery, complaints, and customer profiles could all be kept together using customer case papers. Delivery status is linked to driver, vehicle, hub, and incident data in service event documentation. While app interaction documents provide a framework for platform-generated event records, route exception documents facilitated targeted analysis of manual route overrides. Relational and NoSQL technologies should be combined to provide an integrated analytics environment at NorthStar.

Orders, deliveries, drivers, cars, hubs, and incidents are examples of structured data that should continue to be accessible for SQL-based analysis. However, MongoDB should be used to store flexible records like customer case histories, app events, route exceptions, complaint history, and service event sequences.

Additionally, NorthStar needs to enhance hub-level monitoring. Problems with route planning, personnel, dispatch delays, traffic, or vehicle availability should be looked into in hubs with high failure rates, longer average delivery durations, or high route exception counts.

High override numbers could be a sign of inadequate route allocation, real-time interruption, or driver-level route adjustments, therefore manual route overrides should be actively watched. To ascertain whether route modifications are connected to unsuccessful or delayed services, route override data should be connected with delivery status and customer complaints.

It is also necessary to assess vehicle maintenance and driver training. Drivers with a high failure rate could need more assistance, a different route, or more training. Priority maintenance reviews should be given to vehicles with greater incident rates or poorer battery health.

Lastly, as part of its database strategy, NorthStar ought to include indexing and query optimization. Indexed fields like customer_id, order_id, delivery_status,

manual_route_override_count, and event_type will make it easier for the company to obtain operational information as data volume grows.

Area	Recommendation	Reason
Data integration	Connect orders, deliveries, complaints, hubs, drivers, vehicles, and incidents	Helps identify root causes instead of isolated symptoms
Hub performance	Monitor failure rate, delivery duration, and route exceptions by hub	Identifies underperforming locations
Route planning	Track manual route override behaviour	Helps detect routing issues or real-time disruption
Customer experience	Link complaints with delivery outcomes and service types	Helps understand dissatisfaction causes
Driver performance	Review drivers with high failure rates or lower training scores	Supports targeted training and operational support
Vehicle maintenance	Prioritise vehicles with high incidents or lower battery health	Reduces downtime and operational disruption
MongoDB design	Store customer cases, app events, and route exceptions as documents	Supports flexible nested records
Optimisation	Use indexes and explain plans	Improves query performance as data volume grows

10. Conclusion

Using Python data processing, SQL within R, R analytics and visualization, MongoDB Atlas, PyMongo, CRUD operations, aggregation queries, indexing, and explain plans, this project examined NorthStar Urban Mobility and Logistics.

According to the investigation, fragmented data, delivery errors, customer complaints, manual route overrides, hub-level variations, driver performance, vehicle condition, and app interaction data are all connected to NorthStar's service issues. The datasets were cleaned, transformed, merged, and analyzed using Python. Structured operational data was queried and integrated views were created using SQL within R. Patterns in delivery results, complaints, hub performance, route behavior, and service reliability were presented with the aid of R analytics and visualization.

A NoSQL database that better supports flexible and nested operational records was created using MongoDB Atlas. The document model made it possible to store customer cases, service events, route exceptions, and app interactions in a way that accurately represents the operational complexity of NorthStar. MongoDB can facilitate both operational case management and analytical reporting, as demonstrated via CRUD operations and aggregate queries.

SQL and MongoDB indexing were used to illustrate query optimization and explain plans. This demonstrated how NorthStar could increase query efficiency as data volume increases. Overall, the study shows how relational analysis and document-based NoSQL design may be used to promote improved decision-making in a complex urban mobility and logistics organization.

11. References

MongoDB. (2026) MongoDB Atlas, PyMongo and Indexes Documentation. Available at: <https://www.mongodb.com/docs/> (Accessed: 10 May 2026).

OpenAI. (2026) ChatGPT. Available at: <https://chatgpt.com/> (Accessed: 10 May 2026).

Pandas. (2026) Pandas Documentation. Available at: <https://pandas.pydata.org/docs/> (Accessed: 10 May 2026).

R Core Team. (2026) R: A Language and Environment for Statistical Computing. Available at: <https://www.r-project.org/> (Accessed: 10 May 2026).

Wickham, H. et al. (2026) ggplot2 Documentation. Available at: <https://ggplot2.tidyverse.org/> (Accessed: 10 May 2026).

University of West London. (2026) Case Study, Assignment Brief and Coursework Guidelines: NorthStar Urban Mobility and Logistics. University of West London.

12. GitHub Repository Link

GitHub Repository Link:

<https://github.com/lamJeril/northstar-databases-analytics>